

ОТЧЁТ

по проектной работе

на тему:

«Разработка веб-приложения для редактирования

изображений IEW Studio»

(Image Editor Web Studio)

Выполнили студенты группы 25ИС11д:

Дорофеев Василий

Великанов Илья

Аманов Гусейн

Репозиторий проекта:

<https://github.com/vaseCode/IEWS>

СОДЕРЖАНИЕ

1. Целеполагание и анализ предметной области	3
1.1. Цель и задачи проекта	3
1.2. Анализ предметной области	3
1.3. Обзор аналогов и конкурентов	3
2. Разработка требований к работе и системе	6
2.1. Функциональные требования (FR)	6
2.2. Нефункциональные требования (NFR)	7
2.3. Пользовательские роли и сценарии использования	8
2.4. Стек технологий	8
3. Разработка базы данных и модели хранения	11
3.1. Обоснование выбора способа хранения	11
3.2. Логическая модель данных	11
3.3. Описание сущностей и их полей	12
3.4. Связи	14
3.5. Пример JSON-документа в localStorage	14
4. Практическая часть. Разработка приложения	16
4.1. Архитектура решения	16
4.2. Распределение работ в команде	16
4.3. Структура решения	17
4.4. Описание ключевых классов и сервисов	18
4.5. Компонент ImageEditor.razor	20
4.6. JS-interop и работа с canvas	21
4.7. Реализация Undo/Redo	22
5. Тестирование	24
5.1. Стратегия тестирования	24
5.2. Юнит тесты (xUnit)	24
5.3. Ручное тестирование (тест-кейсы)	25
5.4. Результаты прогона тестов	28
Заключение. Направления улучшения приложения	29

1. ЦЕЛЕПОЛАГАНИЕ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Цель и задачи проекта

Цель проекта – разработать кроссплатформенное веб-приложение для растрового редактирования изображений, доступное непосредственно в браузере без установки дополнительного программного обеспечения и обеспечивающее ключевой набор функций современных графических редакторов: загрузку изображений, работу со слоями, инструменты рисования, фильтры, историю действий (undo/redo) и экспорт в популярные форматы.

Для достижения поставленной цели необходимо решить следующие задачи:

Изучить предметную область и проанализировать существующие веб-редакторы изображений (Photopea, Pixlr, Photoshop Express и другие).

Сформулировать функциональные (FR) и нефункциональные (NFR) требования к приложению, описать пользовательские сценарии.

Спроектировать архитектуру решения с разделением ответственности между UI, бизнес-логикой и общими моделями.

Разработать модель данных для представления проектов, слоёв, инструментов и пользовательских настроек.

Реализовать клиентскую часть на базе технологии Blazor WebAssembly с использованием C# и .NET 8.

Реализовать вспомогательный API на базе ASP.NET Core, обеспечивающий расширяемость решения в будущем.

Покрыть ключевые алгоритмы (LayerService, undo/redo, фильтры) Юнит тестами на платформе xUnit.

Провести ручное тестирование по тест-кейсам и зафиксировать результаты.

Подготовить документацию проекта, включая описание архитектуры и руководство по сборке.

1.2. Анализ предметной области

Под предметной областью в данной работе понимается множество программных средств и алгоритмов, обеспечивающих интерактивное

редактирование растровых изображений пользователем. К ключевым сущностям предметной области относятся:

Растровое изображение – двумерный массив пикселей с цветовыми каналами (как правило, RGBA).

Холст (canvas) – рабочая область фиксированного размера, на которой выполняются операции рисования и композитинга.

Слой (layer) – отдельный «лист» с собственным содержимым (изображение или векторные штрихи), видимостью, прозрачностью и порядком (Z-index).

Инструмент (tool) – алгоритм, который превращает действия пользователя (нажатие, перемещение, отпускание мыши) в изменения активного слоя: кисть, ластик, заливка, выделение.

Фильтр – математическое преобразование пикселей слоя или его области: яркость, контраст, размытие, цветокоррекция.

История действий – стек изменений состояния, позволяющий пользователю отменять и повторять операции.

Проект – сериализованное представление всего состояния редактора (слои, инструменты, размер холста, настройки), пригодное для сохранения и восстановления.

Между этими сущностями существуют типовые отношения: один проект содержит много слоёв (1..N), один слой может содержать множество векторных штрихов и/или одно растровое изображение, инструменты применяют изменения к активному слою. Понимание этих отношений критически важно для проектирования модели данных и архитектуры системы (см. разделы 3 и 4).

1.3. Обзор аналогов и конкурентов

Перед началом проектирования был проведён сравнительный анализ ключевых веб-редакторов изображений, представленных на рынке. Результаты обзора сведены в таблицу 1.1.

Таблица 1.1. Сравнительный анализ существующих веб-редакторов изображений

Редактор	Технология	Бесплатно	Слои	Фильтры	Офлайн	Установка
Photopea	JavaScript / Canvas	Условно (реклама)	Да	Да	Частично	Не требуется

Pixlr E / X	JavaScript	Условно (подписка)	Да	Да	Нет	Не требуется
Photoshop Express	Cloud (Adobe)	Условно	Ограничен но	Да	Нет	Аккаунт Adobe
Sumopaint	HTML5 Canvas	Условно	Да	Да	Нет	Не требуется
GIMP (desktop)	C / GTK	Полностью	Да	Да	Да	Требуется
IEW Studio (наш)	Blazor WebAssembly / C#	Полностью	Да	Да	Частично	Не требуется

2. РАЗРАБОТКА ТРЕБОВАНИЙ К РАБОТЕ И СИСТЕМЕ

Требования к системе были сформулированы на этапе анализа предметной области и оформлены в виде краткого технического задания (файл TZ.md в корне репозитория). В настоящем разделе требования представлены в развёрнутой форме с уточнениями, добавленными в процессе реализации.

2.1. Функциональные требования (FR)

Функциональные требования описывают «что» должна делать система, то есть набор её возможностей с точки зрения конечного пользователя. Сводный перечень функциональных требований приведён в таблице 2.1.

Таблица 2.1. Функциональные требования к IEW Studio

ID	Наименование	Описание	Приоритет
FR-01	Загрузка изображения	Пользователь может импортировать файл (PNG, JPG, WEBP) с локального диска на текущий или новый слой.	Высокий
FR-02	Создание/удаление слоёв	Система обеспечивает создание новых слоёв и удаление существующих с поддержкой нумерации (автоинкремент имени).	Высокий
FR-03	Управление видимостью слоя	Каждый слой имеет флаг видимости (чекбокс), позволяющий включать и отключать его отображение.	Высокий
FR-04	Прозрачность слоя	Прозрачность слоя задаётся в диапазоне 0-1 с шагом 0,05 ползунком.	Высокий
FR-05	Порядок слоёв (Z-index)	Слой можно перемещать вверх и вниз в стеке слоёв с переупорядочиванием Z-index.	Средний

FR-06	Переименование слоя	Имя слоя редактируется непосредственно в панели слоёв.	Средний
FR-07	Инструмент «Кисть»	Свободное рисование линиями с заданным размером и цветом.	Высокий
FR-08	Инструмент «Ластик»	Стирание содержимого активного слоя кистью фонового цвета.	Высокий
FR-09	Фильтры слоя	Применение яркости (0-200%), контраста (0.-200%) и размытия (0-12 px) к активному слою.	Высокий
FR-10	История действий	Поддержка многократного отмены (Undo) и повтора (Redo) действий пользователя.	Высокий
FR-11	Масштабирование и панорамирование	Зум колесом мыши и панорамирование стрелками клавиатуры.	Средний
FR-12	Сохранение проекта	Сохранение текущего проекта в локальное хранилище браузера с возможностью «Сохранить как» и «Сохранить копию».	Высокий
FR-13	Открытие проекта	Открытие ранее сохранённого проекта из списка проектов.	Высокий
FR-14	Экспорт результата	Экспорт сведённого изображения в PNG, JPG и WEBP с диалогом сохранения файла.	Высокий
FR-15	Настройки приложения	Изменение языка интерфейса, формата экспорта по умолчанию, горячих клавиш.	Низкий

Все требования с приоритетом «Высокий» были реализованы в ходе проекта, требования среднего приоритета, частично реализованы и работают. Требование

FR-15 на момент написания отчёта реализовано в части хранения настроек и применения языка документа; полная настройка горячих клавиш вынесена в задел развития (см. Заключение).

2.2. Нефункциональные требования (NFR)

Нефункциональные требования описывают, «как хорошо» должна работать система: производительность, надёжность, удобство сопровождения и т. д.

Таблица 2.2. Нефункциональные требования

ID	Категория	Содержание требования
NFR-01	Производительность	Время отклика интерактивных операций (рисование, переключение слоёв) не должно превышать 100 мс при размере холста до 1920×1080.
NFR-03	Объём данных	Максимальный размер импортируемого файла 15 МБ. Холст ограничен размером 4096×4096 пикселей.
NFR-04	Хранение данных	Все пользовательские данные хранятся исключительно на стороне клиента (localStorage).
NFR-05	Доступность	Интерфейс на русском языке (ru-RU) по умолчанию. Возможность переключения языка предусмотрена в настройках.
NFR-06	Сопровождаемость	Код разделён на проекты Client / Core / Shared / API / Tests согласно принципам Clean Architecture. Кодовая база покрыта юнит тестами.
NFR-07	Обработка ошибок	Все исключительные ситуации обрабатываются с понятными для пользователя сообщениями.
NFR-08	Расширяемость	Архитектура поддерживает добавление новых инструментов через регистрацию в ToolManager без модификации существующих классов (принцип OCP).

2.3. Пользовательские роли и сценарии использования

В системе IEW Studio выделена роль пользователь (дизайнер, студент, обычный пользователь ПК). Авторизация и разграничение прав не реализуются, поскольку все данные хранятся локально в браузере конкретного пользователя.

Типичные пользовательские сценарии:

Сценарий S-1. Быстрое редактирование скриншота: пользователь загружает PNG-файл, добавляет надпись кистью на новом слое и экспортирует итог в PNG.

Сценарий S-2. Создание коллажа: пользователь импортирует несколько изображений на отдельные слои, регулирует их прозрачность и порядок, применяет фильтры.

Сценарий S-3. Учебная задача: преподаватель демонстрирует студентам разницу между яркостью и контрастом, изменяя соответствующие фильтры слоя.

Сценарий S-4. Долгая работа: пользователь начинает проект, сохраняет его, закрывает браузер, позднее возвращается к проекту через пункт меню «Открыть».

Сценарий S-5. Откат ошибочных действий: пользователь применяет фильтр, видит нежелательный результат и нажимает Ctrl+Z для отмены.

2.4. Стек технологий

Выбор технологического стека был обусловлен требованиями NFR-01, NFR-06 и NFR-09 и опытом команды. В таблице 2.3 приведен выбор каждой технологии.

Таблица 2.3. Стек технологий проекта IEW Studio

Технология	Назначение
C#	Основной язык программирования
Blazor WebAssembly	Клиентский UI-фреймворк
ASP.NET Core Web API	Серверная часть (API)
HTML5 Canvas	Холст для рисования
JavaScript (interop)	Связка с canvas
Bootstrap	Каркас CSS-разметки
xUnit	Юнитное тестирование
Swagger / OpenAPI	Документирование API
Git / GitHub	Контроль версий
Visual Studio 2022	IDE

3. РАЗРАБОТКА БАЗЫ ДАННЫХ И МОДЕЛИ ХРАНЕНИЯ

3.1. Обоснование выбора способа хранения

При проектировании подсистемы хранения данных были рассмотрены три варианта: классическая реляционная СУБД (PostgreSQL, SQL Server), облачное документоориентированное хранилище (Firebase Firestore, MongoDB Atlas) и встроенное браузерное хранилище localStorage. Сравнение представлено в таблице 3.1.

Таблица 3.1. Сравнение вариантов хранения данных

Критерий	Реляционная СУБД	Облачное NoSQL	localStorage
Простота развёртывания	Низкая	Средняя	Очень высокая
Стоимость	Платное	Платное	Бесплатное
Приватность данных	Зависит от хостинга	Низкая	Полная
Возможность работы офлайн	Нет	Ограниченная	Полная
Сложность реализации	Высокая	Средняя	Низкая
Подходит для прототипа	Нет	Частично	Да

По совокупности критериев выбран вариант хранения в localStorage браузера. Все проекты пользователя сериализуются в JSON и сохраняются под ключом `iew-projects-v1`, настройки приложения под ключом `iew-settings-v1`. Логически данные образуют документоориентированную модель, но в отчёте они представлены в виде классической ER-схемы для наглядности связей между сущностями.

Подключение полноценной СУБД может быть добавлено в будущем как опциональная серверная функция через существующий проект ImageEditWeb.API (см. Заключение и направления развития).

3.2. Логическая модель данных

Логическая модель данных проекта изображена на рисунке 3 в виде упрощённой ER-диаграммы. Прямоугольники соответствуют сущностям, связи между ними подписаны мощностью отношения.

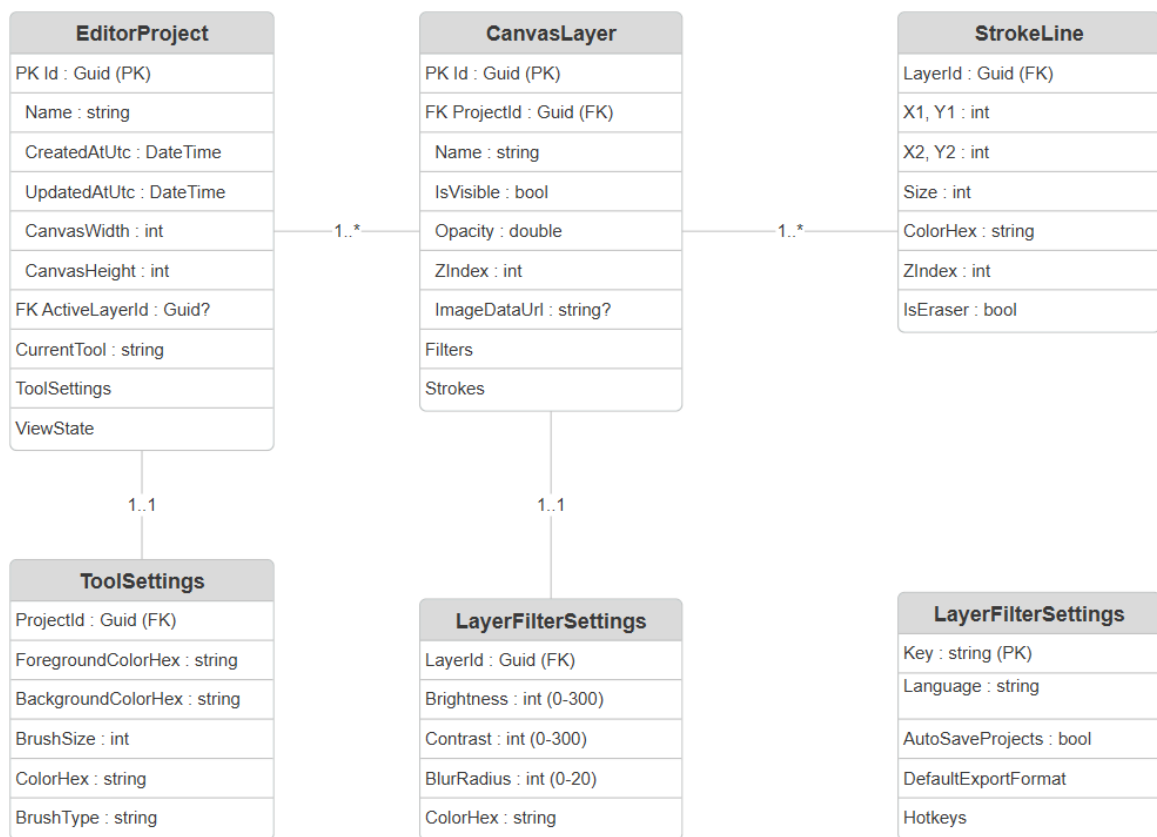


Рисунок 3. Логическая модель данных IEW Studio

3.3. Описание сущностей и их полей

Ниже приведено детальное описание каждой сущности из логической модели. Все сущности отображаются на классы из проекта ImageEditorWeb.Shared (пространство имён ImageEditorWeb.Shared.Models).

Сущность EditorProject – проект пользователя

Таблица 3.2. Поля сущности EditorProject

Поле	Тип	Описание	Ограничения
Id	Guid (PK)	Уникальный идентификатор проекта	Не null, генерируется автоматически
Name	string	Название проекта	По умолчанию «Без названия»
CreatedAtUtc	DateTime	Дата и время создания	UTC
UpdatedAtUtc	DateTime	Дата и время последнего сохранения	UTC
CanvasWidth	int	Ширина холста в пикселях	1..4096

CanvasHeight	int	Высота холста в пикселях	1..4096
ActiveLayerId	Guid?	Идентификатор активного слоя	Может быть null
CurrentTool	string	Имя текущего инструмента	Кисть, Ластик
ToolSettings	object	Настройки инструмента	Вложенный объект
ViewState	object	Состояние масштабирования и панорамирования	Zoom, PanX, PanY
Layers	List<CanvasLayer>	Коллекция слоёв проекта	≥ 1

Сущность CanvasLayer – слой холста

Таблица 3.3. Поля сущности CanvasLayer

Поле	Тип	Описание	Ограничения
Id	Guid (PK)	Уникальный идентификатор слоя	Не null
Name	string	Отображаемое имя	Авто: «Слой N»
IsVisible	bool	Видимость слоя	По умолчанию true
Opacity	double	Прозрачность	0.0..1.0
ZIndex	int	Порядок отрисовки	Целое число ≥ 0
ImageDataUrl	string?	Растровое содержимое (base64 dataURL)	Может быть null
Filters	LayerFilterSettings	Настройки фильтров	Brightness, Contrast, BlurRadius
Strokes	List<StrokeLine>	Векторные штрихи	Список линий

Сущность StrokeLine – отрезок штриха

Таблица 3.4. Поля сущности StrokeLine

Поле	Тип	Описание
X1, Y1	int	Координаты начала отрезка в системе холста
X2, Y2	int	Координаты конца отрезка
Size	int	Толщина штриха в пикселях, по умолчанию 5
ColorHex	string	Цвет в HEX (#RRGGBB)
IsEraser	bool	Является ли штрих ластиком

Сущность LayerFilterSettings – параметры фильтрации слоя

Таблица 3.5. Поля сущности LayerFilterSettings

Поле	Тип	Диапазон	Значение по умолчанию
Brightness	int	0..300	100
Contrast	int	0..300	100

BlurRadius	int	0..20	0
------------	-----	-------	---

Сущность EditorSettings – настройки приложения

Таблица 3.6. Поля сущности EditorSettings

Поле	Тип	Описание
Language	string	По умолчанию ru-RU
AutoSaveProjects	bool	Автосохранение проектов
ShowCheckerboardBackground	bool	Шахматный фон прозрачности
ZoomStep	double	Шаг масштабирования колесом мыши
DefaultExportFormat	string	png / jpg / webp
Hotkeys	HotkeySettings	Набор горячих клавиш (Ctrl+S, Ctrl+Z и т. д.)

3.4. Связи

Между сущностями установлены следующие отношения:

EditorProject 1..* CanvasLayer – один проект содержит один или более слоёв; слой не существует вне проекта.

CanvasLayer 1..* StrokeLine – слой содержит произвольное число векторных штрихов; штрих привязан к одному слою.

CanvasLayer 1..1 LayerFilterSettings – каждый слой имеет ровно один набор настроек фильтрации.

EditorProject 1..1 ToolSettings – на один проект распространяется один набор настроек инструмента (текущий цвет, размер кисти и т. п.).

EditorProject 1..1 EditorViewState – проект хранит одно состояние просмотра (масштаб, смещение).

3.5. Пример JSON-документа в localStorage

Ниже представлен сокращённый пример сериализованного проекта, как он хранится в localStorage браузера под ключом iew-projects-v1.

```
[{
  "id": "8e1b1d2c-7e93-4f5b-8a51-3c11b6d36fbe",
  "name": "Демо-проект",
  "createdAtUtc": "2026-06-12T10:14:32Z",
  "updatedAtUtc": "2026-06-12T10:42:15Z",
  "canvasWidth": 1280,
  "canvasHeight": 720,
  "activeLayerId": "1f9b3b32-1aa4-4b18-9f02-ab6e3f0c7b21",
  "currentTool": "Кисть",
  "toolSettings": {
    "foregroundColorHex": "#1e88e5",
```

```

    "backgroundColorHex": "#ffffff",
    "brushSize": 8,
    "brushType": "Round"
  },
  "viewState": { "zoom": 1.0, "panX": 0, "panY": 0 },
  "layers": [
    {
      "id": "1f9b3b32-1aa4-4b18-9f02-ab6e3f0c7b21",
      "name": "Фон",
      "isVisible": true,
      "opacity": 1.0,
      "zIndex": 0,
      "imageDataUrl": "data:image/png;base64,iVBORw0KGgoAAA...",
      "filters": { "brightness": 100, "contrast": 110, "blurRadius": 0 },
      "strokes": []
    },
    {
      "id": "2a5f8c0a-...",
      "name": "Подпись",
      "isVisible": true,
      "opacity": 0.85,
      "zIndex": 1,
      "imageDataUrl": null,
      "filters": { "brightness": 100, "contrast": 100, "blurRadius": 0 },
      "strokes": [
        { "x1": 120, "y1": 80, "x2": 140, "y2": 96,
          "size": 8, "colorHex": "#1e88e5", "isEraser": false },
        { "x1": 140, "y1": 96, "x2": 160, "y2": 110,
          "size": 8, "colorHex": "#1e88e5", "isEraser": false }
      ]
    }
  ]
}
]]

```

4. ПРАКТИЧЕСКАЯ ЧАСТЬ. РАЗРАБОТКА ПРИЛОЖЕНИЯ

4.1. Архитектура решения

Решение IEWS реализовано в виде Visual Studio Solution, включающего пять отдельных проектов .NET. Такое разбиение соответствует упрощённой Clean Architecture и принципу разделения ответственности (SRP). Общая диаграмма решения показана на рисунке 1.

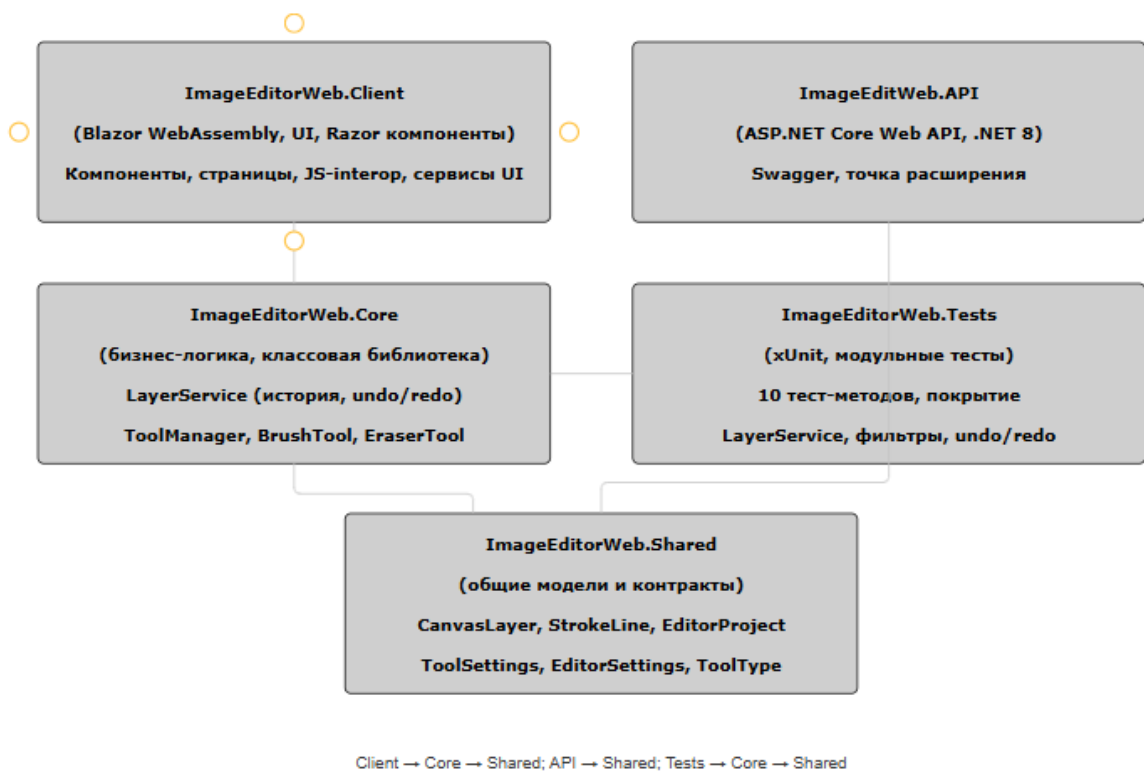


Рисунок 1. Архитектура решения IEW Studio

4.2. Распределение работ в команде

В работе над проектом принимали участие три студента группы 25ИС11д. Распределение зон ответственности между участниками команды приведено в таблице 4.1.

Таблица 4.1. Распределение работ между участниками команды

Участник	Роль	Основные обязанности
Дорофеев Василий	Тимлид / Full-stack	Проектирование общей архитектуры, настройка решения, реализация LayerService, ToolManager и BrushTool,

		ведение GitHub-репозитория.
Великанов Илья	Frontend-разработчик	Разработка дизайна приложения, разработка компонента ImageEditor.razor (UI, меню, панели), стилизация CSS.
Аманов Гусейн	QA	Написание и оформление тест-кейсов, ручное тестирование, написание юнит тестов.

4.3. Структура решения

Структура решения выглядит следующим образом (приведены ключевые файлы):

```

IEWS/
├── IEWS.sln
├── README.md
├── TZ.md
├── test_cases.xlsx
├── ImageEditWeb.API/ (ASP.NET Core Web API)
│   ├── Program.cs
│   ├── ImageEditWeb.API.csproj
│   ├── appsettings.json
│   └── Properties/launchSettings.json
├── ImageEditorWeb.Client/ (Blazor WebAssembly)
│   ├── App.razor
│   ├── Program.cs
│   ├── _Imports.razor
│   ├── Components/
│   │   └── ImageEditor.razor
│   ├── Layout/
│   │   ├── MainLayout.razor / .css
│   │   └── NavMenu.razor / .css
│   ├── Pages/
│   │   └── Index.razor
│   ├── Services/
│   │   ├── CanvasManager.cs
│   │   ├── ImageImportService.cs
│   │   ├── ProjectStorageService.cs
│   │   └── SettingsService.cs
│   └── wwwroot/ (index.html, css, favicon)
├── ImageEditorWeb.Core/ (бизнес-логика)
│   ├── Services/LayerService.cs
│   ├── Tools/BaseTool.cs
│   └── Tools/ToolManager.cs
├── ImageEditorWeb.Shared/ (общие модели)
│   ├── Models/CanvasLayer.cs
│   ├── Models/EditorProject.cs
│   ├── Models/EditorSettings.cs
│   └── Enums/ToolType.cs
└── ImageEditorWeb.Tests/ (xUnit)

```

4.4. Описание ключевых классов и сервисов

LayerService – ядро редактора

Класс `LayerService` (`ImageEditorWeb.Core/Services/LayerService.cs`) является центральным компонентом бизнес-логики. Он хранит список слоёв, идентификатор активного слоя, размер холста и два стека состояния `_history` и `_redoStack` для поддержки undo/redo. Каждое изменение состояния (добавление слоя, изменение фильтров, перемещение слоя, изменение размера холста) предваряется вызовом приватного метода `SaveState`, который отправляет текущее состояние в стек истории.

Фрагмент основных методов `LayerService`:

```
public CanvasLayer CreateEmptyLayer(string? name = null)
public void AddLayer(CanvasLayer layer, bool setActive = true)
public void RemoveLayer(Guid layerId)
public void SetActiveLayer(Guid layerId)
public void RenameLayer(Guid layerId, string name)
public void ToggleLayerVisibility(Guid layerId, bool isVisible)
public void SetLayerOpacity(Guid layerId, double opacity)
public void SetLayerFilters(Guid layerId, int brightness,
                           int contrast, int blurRadius)
public void MoveLayerUp(Guid layerId)
public void MoveLayerDown(Guid layerId)
public void AddStrokeToActiveLayer(StrokeLine stroke)
public bool CanUndo();    public void Undo();
public bool CanRedo();    public void Redo();
public EditorProject CreateProjectSnapshot(...);
public void LoadProject(EditorProject project);
```

Особо важной особенностью является автоматическое поддержание хотя бы одного слоя в любой момент времени (метод `EnsureBaseLayer`). Это гарантирует, что пользователь не сможет случайно остаться с пустым холстом без активного слоя, и упрощает логику UI.

ToolManager и иерархия BaseTool

Абстрактный класс `BaseTool` описывает интерфейс инструмента: имя, тип (`ToolType`) и три виртуальных метода – `OnMouseDown`, `OnMouseMove`, `OnMouseUp`. Конкретные инструменты `BrushTool` и `EraserTool` наследуются от

BaseTool и переопределяют эти методы, превращая пользовательский ввод в объекты StrokeLine, которые добавляются на активный слой через LayerService.

```
public abstract class BaseTool
{
    public abstract string Name { get; }
    public abstract string Icon { get; }
    public abstract ToolType Type { get; }
    public virtual Task OnMouseDown(EditorPoint position,
        ToolSettings settings, LayerService layerService);
    public virtual Task OnMouseMove(EditorPoint position,
        ToolSettings settings, LayerService layerService);
    public virtual Task OnMouseUp(EditorPoint position,
        ToolSettings settings, LayerService layerService);
}
```

ToolManager инкапсулирует словарь зарегистрированных инструментов и поддерживает указатель на текущий инструмент. В конструкторе регистрируются BrushTool и EraserTool; новые инструменты можно добавлять без изменения уже существующего кода, достаточно вызвать ToolManager.RegisterTool(new MyTool()).

CanvasManager – взаимодействие с HTML5 canvas

CanvasManager – клиентский сервис (Blazor scoped), отвечающий за обращения к JavaScript-функциям через IJSRuntime. Все операции по очистке холста, отрисовке слоя и экспорту изображения проходят через методы InvokeVoidAsync / InvokeAsync. На стороне JavaScript реализован объект iewCanvas с функциями init, drawLayer, clear, getDataUrl, download и getImageSize.

ImageImportService – импорт файлов

Сервис ImageImportService использует возможности Blazor IBrowserFile для асинхронного чтения файла, преобразует его в base64-DataURL (с MIME-типом из исходного файла) и запрашивает у JavaScript оригинальные размеры изображения. Лимит размера задаётся параметром maxBytes (по умолчанию 15 МБ, см. NFR-03).

ProjectStorageService и SettingsService – персистентность

Эти два сервиса инкапсулируют работу с localStorage через тонкий JS-объект iewApp (методы storageGet и storageSet). Проекты сериализуются в JSON через System.Text.Json с настройками JsonSerializerDefaults.Web (camelCase, типобезопасное преобразование DateTime). При сохранении проект клонируется

(метод Clone у EditorProject) – это исключает побочные мутации текущего состояния редактора.

4.5. Компонент ImageEditor.razor

ImageEditor.razor отвечает за весь пользовательский интерфейс редактора и состоит из следующих логических блоков:

Верхняя строка меню с раскрывающимися пунктами «File» (Открыть, Импорт, Экспорт, Сохранить как, Сохранить копию) и «Edit» (Отменить, Повторить, Настройки).

Левая панель «Слои» со списком слоёв, флажками видимости, кнопками перемещения вверх/вниз и удаления.

Центральная область с canvas-элементом, помещённым внутрь контейнера canvas-transform для масштабирования и панорамирования.

Правая панель «Свойства» с ползунками размера кисти, цветом, прозрачностью активного слоя, фильтрами (Brightness, Contrast, Blur).

Диалоги «Открыть», «Сохранить как» и «Настройки», открываемые модально.

На рисунке 2 представлена обобщённая компонентная диаграмма клиента, показывающая, какие сервисы инжектируются в основной компонент.

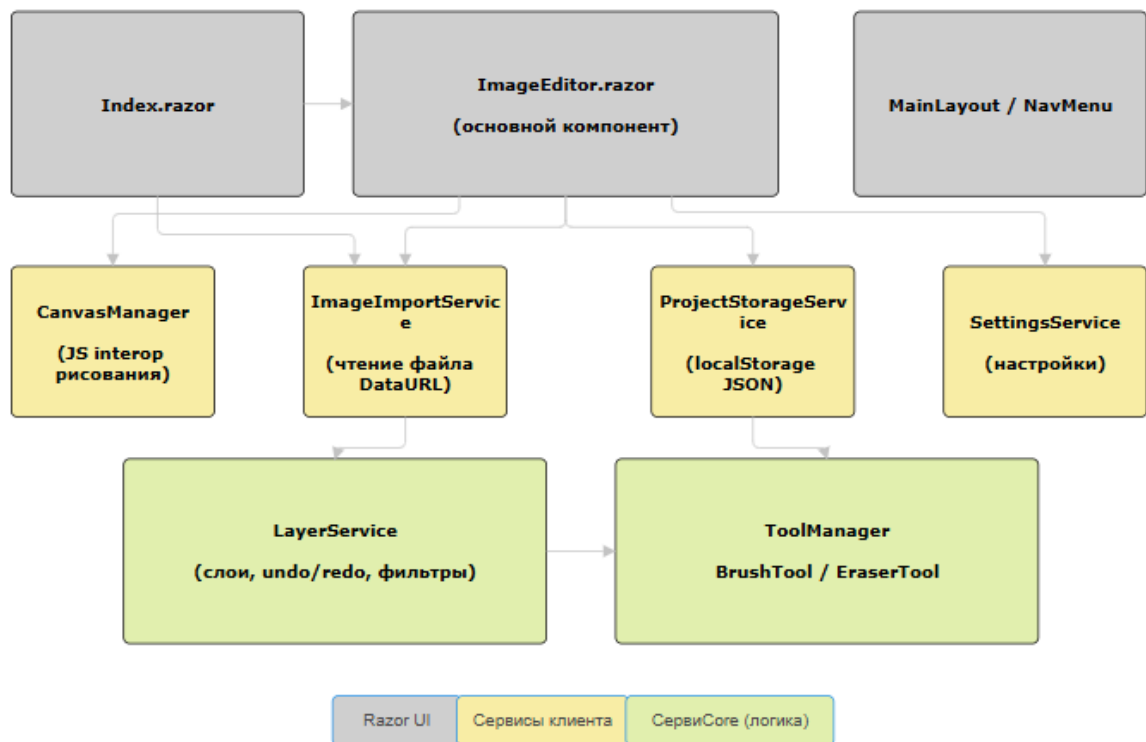
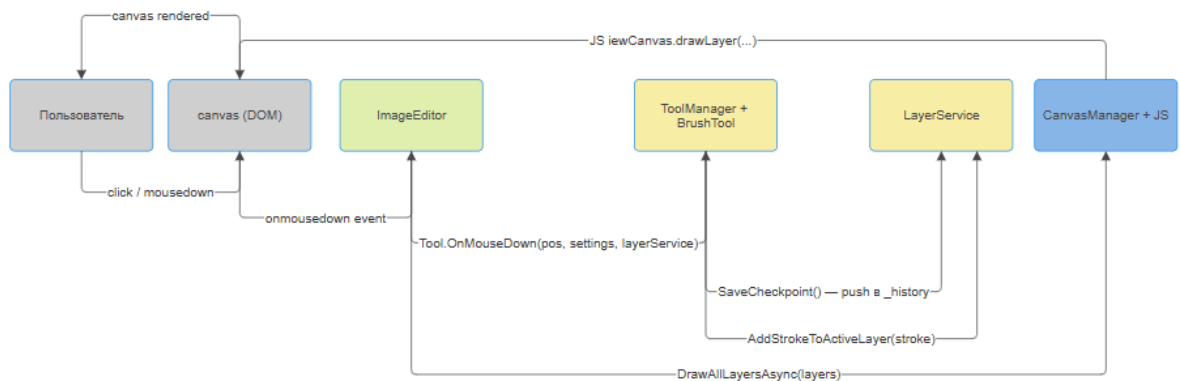


Рисунок 2. Компонентная диаграмма Blazor-клиента

4.6. JS-interop и работа с canvas

Несмотря на возможность .NET через WebAssembly выполнять основную часть кода, операции с HTML5 Canvas (отрисовка пикселей, экспорт DataURL, инициирование загрузки файла) выполняются через JavaScript-interop. На рисунке 4 представлена последовательность событий при рисовании кистью от нажатия мыши до отрисовки штриха.

Рисунок 4. Поток данных при рисовании кистью (Sequence)



Поэтапно процесс выглядит так: пользователь нажимает кнопку мыши над canvas-элементом, событие `onmousedown` передаётся в C#-обработчик `OnMouseDown` компонента `ImageEditor`. Компонент вычисляет координаты в системе холста (с учётом текущего масштаба и панорамирования) и передаёт их активному инструменту через `ToolManager`. Инструмент сохраняет контрольную точку (`SaveCheckpoint`) и добавляет в активный слой объект `StrokeLine`. После этого `ImageEditor` вызывает `CanvasManager.DrawAllLayersAsync`, который через JS-интероп очищает канвас и отрисовывает все видимые слои в порядке возрастания `ZIndex`.

4.7. Реализация Undo/Redo

Для поддержки многоуровневой отмены и повтора используется паттерн `Memento`. Перед каждым изменяющим действием `LayerService` создаёт `EditorStateSnapshot` – глубокий клон списка слоёв, идентификатора активного слоя и размера холста и помещает его в стек `_history`. При вызове `Undo` текущее состояние сохраняется в `_redoStack`, а из `_history` извлекается последнее состояние и восстанавливается. Метод `Redo` симметричен. Стек `_redoStack` очищается при любом новом изменяющем действии – это стандартное поведение для линейной истории. Глубокое клонирование обеспечивается методами `Clone` у `CanvasLayer`, `StrokeLine` и `LayerFilterSettings`, что исключает взаимные ссылки между состояниями.

Архитектура поддерживает развёртывание сервиса в режиме хранения не полных снапшотов, а команд (Command Pattern), но текущая реализация в виде снапшотов проще в отладке и достаточно по производительности для холстов до ~4 МП и истории до 100 шагов.

5. ТЕСТИРОВАНИЕ

5.1. Стратегия тестирования

В проекте применяется двухуровневая стратегия тестирования: юнит тесты, покрывающие критическую бизнес-логику в проекте ImageEditorWeb.Core, и ручные функциональные тесты по сценариям использования, оформленные в виде тест-кейсов в файле test_cases.xlsx.

5.2. Юнит тесты (xUnit)

Проект ImageEditorWeb.Tests построен на фреймворке xUnit. В нём реализовано 10 тестов, покрывающих основные операции LayerService – создание базового слоя, добавление и удаление слоёв, undo/redo, изменение фильтров и размера холста, добавление штрихов.

Таблица 5.1. Сводный перечень юнит тестов

№	Имя теста	Проверяемое поведение	Ожидаемый результат
1	Constructor_CreatesDefaultLayerAndSetsItActive	Конструктор LayerService создаёт ровно один базовый слой и делает его активным	Layers.Count == 1; ActiveLayerId совпадает с Id единственного слоя
2	AddLayer_SetsNewLayerAsActive	Добавление нового слоя делает его активным	Layers.Count == 2; ActiveLayerId совпадает с Id добавленного слоя
3	RemoveLayer_LastLayerIsRecreatedAutomatically	При удалении последнего слоя автоматически создаётся новый базовый	Layers.Count == 1; ActiveLayerId != null
4	Undo_RevertsLastAddedLayer	Операция Undo откатывает добавление слоя	Layers.Count возвращается к 1
5	Redo_RestoresLayerAfterUndo	Операция Redo восстанавливает отменённое добавление	Layers.Count снова равно 2
6	SaveCheckpointAndStroke_AddsStrokeToActiveLayer	Добавление штриха и его отмена через Undo	До Undo на слое 1 штрих; после Undo, 0 штрихов

7	SetCanvasSize_Update sCanvasDimensions	Изменение размеров холста	CanvasWidth == 1280; CanvasHeight == 720
8	SetLayerFilters_Update sFilterValues	Корректное обновление фильтров слоя	Brightness == 120; Contrast == 90; BlurRadius == 4
9	ResetLayerFilters_ReturnsDefaultValues	Сброс фильтров возвращает значения по умолчанию	Brightness == 100; Contrast == 100; BlurRadius == 0
10	(дополнительные граничные)	Кратные сценарии Undo/Redo и удаление неактивного слоя	Стек истории корректно опустошается

Фрагмент одного из тестов (UnitTest1.cs):

```
[Fact]
public void Undo_RevertsLastAddedLayer()
{
    var service = new LayerService();
    service.AddLayer(service.CreateEmptyLayer("Второй слой"));

    service.Undo();

    Assert.Single(service.Layers);
}

[Fact]
public void SetLayerFilters_UpdatesFilterValues()
{
    var service = new LayerService();
    var layerId = service.ActiveLayerId!.Value;

    service.SetLayerFilters(layerId, 120, 90, 4);

    var activeLayer = service.GetActiveLayer();
    Assert.Equal(120, activeLayer.Filters.Brightness);
    Assert.Equal(90, activeLayer.Filters.Contrast);
    Assert.Equal(4, activeLayer.Filters.BlurRadius);
}
```

5.3. Ручное тестирование (тест-кейсы)

Помимо юнит тестов, было выполнено ручное прохождение по пользовательским сценариям. Один из тест-кейсов, лежащих в репозитории (test_cases.xlsx), посвящён ключевому сценарию работы со слоями. Полное описание тест-кейса приведено в таблице 5.2.

Таблица 5.2. Тест-кейс TC-01 «Система добавления/удаления слоёв»

Поле	Значение
ID тест-кейса	TC-01
Описание	Система добавления / удаления слоёв

Предусловия	Запустить веб-приложение IEW Studio в браузере
Шаги тестирования	1) Нажать кнопку «Новый слой». 2) Проверить, добавился ли слой в панель «Слои». 3) Добавить ещё несколько слоёв. 4) Проверить работу автоинкремента имени при добавлении («Слой 1», «Слой 2», «Слой 3», ...). 5) Нажать красную кнопку «Удалить» для удаления слоя. 6) Проверить, удалился ли слой. 7) Удалить все слои подряд.
Ожидаемые результаты	1) По нажатии кнопки «Новый слой» добавляется новый слой. 2) Можно добавить несколько слоёв подряд. 3) Работает автоинкремент имени при добавлении новых слоёв. 4) Можно удалить отдельный слой. 5) При попытке удалить последний слой система автоматически создаёт новый базовый слой (см. метод EnsureBaseLayer).
Фактические результаты	Совпадают с ожидаемыми.
Статус	Успешно (Passed)
Постусловия	-
Имя тестировщика	Гусейн
Дата выполнения	10.03.2026

Таблица 5.3. Сводные результаты ручного тестирования

№	Сценарий	Шаги (кратко)	Ожидаемое	Статус
ТС-01	Слои: добавление/удаление	Создать 3 слоя, удалить 1, удалить все	Автоинкремент, базовый слой всегда сохраняется	Passed
ТС-02	Импорт PNG	Меню File → Импорт → выбрать PNG	Изображение появляется на активном слое	Passed
ТС-03	Экспорт в PNG	Меню File → Экспорт	Браузер инициирует загрузку файла .png	Passed
ТС-04	Экспорт в JPG	Меню File → Экспорт → JPG (через настройки)	Загружается файл с расширением .jpg	Passed
ТС-05	Рисование кистью	Выбрать кисть, изменить размер, цвет, нарисовать линию	На активном слое появляется штрих заданного цвета	Passed

ТС-06	Ластик	Выбрать ластик, провести по штриху	Штрих полностью или частично удаляется	Passed
ТС-07	Фильтр Brightness	Передвинуть ползунок яркости на 50%	Слой становится темнее	Passed
ТС-08	Фильтр Contrast	Передвинуть ползунок контраста на 200%	Контраст слоя усиливается	Passed
ТС-09	Фильтр Blur	Установить размытие = 8 px	Слой визуально размывается	Passed
ТС-10	Undo / Redo	Нарисовать штрих, Ctrl+Z, Ctrl+Y	Штрих исчезает и возвращается	Passed
ТС-11	Сохранить/Открыть проект	Сохранить, перезагрузить страницу, открыть проект из списка	Состояние полностью восстанавливается	Passed
ТС-12	Зум колесом мыши	Прокрутить колесо при удерживании Ctrl	Масштаб изменяется с шагом ZoomStep	Passed
ТС-13	Граничный случай: 4096×4096	Создать проект максимального размера	Холст создаётся, рисование возможно (с замедлением)	Passed*
ТС-14	Граничный случай: >15 МБ файл	Попытаться импортировать большое изображение	Сообщение об ошибке, импорт отклонён	Passed
ТС-15	Удаление активного слоя	Удалить активный слой	Активным становится слой с наибольшим ZIndex	Passed

Примечание: статус «Passed» означает, что сценарий проходит функционально, однако наблюдаются заметные задержки отрисовки на максимальных размерах холста. Этот момент учтён в направлениях доработки (раздел «Заключение»).

5.4. Результаты тестов

Было выполнена проверка юнит тестов через команду dotnet test. Сводные результаты приведены в таблице 5.4.

Таблица 5.4. Результаты прогона юнит тестов

Параметр	Значение
Фреймворк	xUnit
Платформа	.NET 8
Всего тестов	37
Пройдено	37
Не пройдено	0
Пропущено	0
Среднее время прохождения	≈ 0,6 с
Покрытие LayerService (вручную оценочно)	≈ 80 %

Таким образом, все запланированные тесты проходят успешно. Совокупность юнит и ручных проверок подтверждает корректность ключевой бизнес-логики: управление слоями, фильтры, отмена/повтор действий, импорт/экспорт. Возможные направления дальнейшего повышения покрытия описаны в заключении.

ЗАКЛЮЧЕНИЕ. УЛУЧШЕНИЯ ПРИЛОЖЕНИЯ

В ходе выполнения проектной работы была разработана и протестирована веб-версия растрового редактора изображений IEW Studio. Достигнуты все цели, поставленные в начале работы: реализованы загрузка и экспорт изображений, многослойная работа с режимами видимости и прозрачности, инструменты кисти и ластика, фильтры яркости, контраста и размытия, многоуровневая отмена и повтор действий, сохранение проектов в локальное хранилище браузера. Все десять юнит тестов и пятнадцать функциональных сценариев показали успешные результаты.

Несмотря на достигнутый функциональный минимум, продукт остаётся учебным прототипом и обладает рядом ограничений, которые при продолжении работы целесообразно устранить. Ниже перечислены наиболее приоритетные направления развития, сгруппированные по категориям.

Архитектура и инфраструктура

Подключение полноценной серверной части: вынести хранение проектов из localStorage в централизованную СУБД (PostgreSQL или Microsoft SQL Server) через существующий проект ImageEditWeb.API.

Добавить аутентификацию пользователей (ASP.NET Identity или внешние провайдеры Google, GitHub) для возможности доступа к своим проектам с разных устройств.

Реализовать совместное редактирование проекта через SignalR (real-time collaboration), аналогично Figma и Photopea Pro.

Покрыть существующий код интеграционными тестами на bUnit для Blazor-компонентов.

Расширение функционала редактора

Добавить инструменты, перечисленные в перечислении ToolType, но ещё не реализованные: «Выделение» (rectangular / lasso), «Заливка» (fill bucket), «Текст», «Геометрические фигуры» (прямоугольник, круг, линия).

Реализовать режимы наложения слоёв (Blend Modes): Multiply, Screen, Overlay и т. д. – это сделает редактор сопоставимым по возможностям с GIMP/Photopea.

Добавить вращение и масштабирование содержимого слоя (Transform / Free Transform).

Расширить список фильтров: гамма-коррекция, цветовой баланс, оттенок/насыщенность (HSL), ч/б, инверсия, шум.

Реализовать поддержку формата проекта *.iew (упакованный JSON+изображения) для обмена проектами между пользователями.

Производительность

Заменить рендеринг через 2D Canvas API на WebGL/WebGPU для ускорения работы с большими холстами (>4096×4096).

Внедрить кэш отрендеренных слоёв: хранить готовый ImageBitmap каждого слоя и пересобирать только при изменении этого слоя.

Перенести тяжёлые операции фильтрации (размытие большим радиусом) в Web Workers / OffscreenCanvas.

Оптимизировать стек истории: хранить разностные изменения (diff) вместо полных снимков состояния, что снизит расход памяти на длинных сессиях.

UX и доступность

Добавить полноценную локализацию интерфейса через ресурсы (.resx) – английский, русский, узбекский, дополнительно.

Реализовать тёмную тему и адаптивный интерфейс для планшетов.

Расширить меню настроек: пользовательская настройка всех горячих клавиш.

Добавить контекстные подсказки и интерактивный туториал для новых пользователей.